

# Coursework: Denoising of Bird Song Recordings Using Autoencoder Model

BY GEWENXIN YU

Email: [ec22214@qmul.ac.uk](mailto:ec22214@qmul.ac.uk)

## 1 Introduction

In the task of bird song detection, it is crucial to consider the impact of environmental noise, as both xeno-canto and Warblr accept user-uploaded recordings, which often include recordings made with smartphones in various environments. These recordings may contain uncertain environmental noises, such as white noise, footsteps, or even sharp sounds that can occur at the beginning of a recording. As Denton et al.[1] described, the presence of significant environmental noise and overlapping vocalizations in the wildlife audio recordings can interfere with accurate classification. Apart from the white noise in the background, another possible source of noise is during the dawn chorus, when many species are singing at the same time.

In the implementation of autoencoder for this coursework, the effective removal of background white noise will be the main objective. Two models, including autoencoder denoising model and a CNN model are used for the evaluation of the final classification task.

## 2 Background

Bird song recognition has become a popular area of audio deep learning in recent years. It's an important area for automated wildlife monitoring and ecological audio archiving.

In 2021, *Rainforest Connection* hosted a competition called *Rainforest Connection Species Audio Detection*<sup>1</sup>, which focused on automating the detection of bird and frog species in a tropical soundscape. It also aimed to identify critical habitat for endangered species and to help prevent illegal deforestation. In terms of wild animal audio databases, *xeno-canto*<sup>2</sup> has made significant contributions by collecting a large number of wild animal sound files from around the world. They also allow users to upload their own recordings of wild animal sounds on their website. Additionally, the 2018 challenge of DCASE<sup>3</sup> also included a task on bird audio detection.

One notable company in the commercial sector is Warblr, based in the UK, which provides a mobile application that allows users to record bird songs in real time and quickly obtain the corresponding bird species names. Other applications, such as BirdNET or Merlin Sound ID, also allow users to identify birds by song.

## 3 Method

### 3.1 Datasets

The Warblrb10k dataset [4] is a crowdsourced dataset containing 10,000 smartphone recordings from across the UK. Each recording is encoded in Microsoft PCM format at a sample rate of 44100 Hz and the average bit per second is 88200. Each recording lasts approximately 10 seconds.

---

1. <https://www.kaggle.com/c/rfcx-species-audio-detection>

2. <https://xeno-canto.org/>

3. <https://dcase.community/challenge2018/task-bird-audio-detection>

Since it is crowdsourced, this dataset is well suited for a binary classification task to detect whether a bird is present in a recording. In addition, the bird recordings in this dataset also contain many complex environmental sounds, such as weather noise, traffic noise or human voices. This makes it ideal for training an autoencoder for noise reduction of audio.

A subset of this dataset `warblrb10k_subset` will be used in this coursework due to computability, computation time and limited GPU resources. As demonstrated in lab3, `maxfilestoload` is set to 100 to prevent long fetches of dataset.

## 3.2 Hardware Requirement

The version of Torch used in this coursework is `1.13.0.dev20220918+cu117`, and the CUDA GPU NVIDIA A40 is available through the university's JupyterHub account.

The NVIDIA A40 has a major version of 8, a minor version of 6, a total memory of 5019MB, and 84 multi-processor counts.

## 3.3 The Autoencoder Model

### 3.3.1 Overview

The autoencoder model is used as a denoising model in this task. For the architecture of this autoencoder model, the output of `summary` method from `Model` class provide a good description:

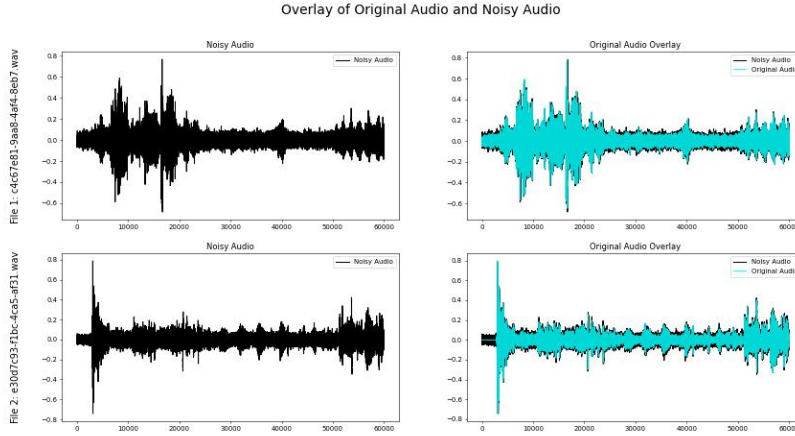
| Layer (type)                         | Output Shape       | Param # |
|--------------------------------------|--------------------|---------|
| input_1 (InputLayer)                 | [(None, 60000, 1)] | 0       |
| conv1d (Conv1D)                      | (None, 60000, 32)  | 128     |
| max_pooling1d (MaxPooling1D)         | (None, 30000, 32)  | 0       |
| conv1d_1 (Conv1D)                    | (None, 30000, 64)  | 6208    |
| max_pooling1d_1 (MaxPooling1D)       | (None, 15000, 64)  | 0       |
| conv1d_transpose (Conv1DTranspose)   | (None, 30000, 64)  | 12352   |
| conv1d_transpose_1 (Conv1DTranspose) | (None, 60000, 64)  | 12352   |
| conv1d_transpose_2 (Conv1DTranspose) | (None, 60000, 32)  | 6176    |
| conv1d_2 (Conv1D)                    | (None, 60000, 1)   | 97      |
| Total params: 37,313                 |                    |         |
| Trainable params: 37,313             |                    |         |
| Non-trainable params: 0              |                    |         |

**Table 1.** The Architecture of the Autoencoder Model

### 3.3.2 Dataset Preparation

For a denoising autoencoder, the input to the encoding layer should normally be audio files within the manually added noise, rather than the original audio files. In the case of the bird recording dataset for this task, although the original recording already contains many types of noise, ranging from artificial noise to background noise, it still requires additional noise to be added. The reason for this is that the denoising model requires a controlled and standardised level of noise in order to effectively learn and remove the unwanted noise from the input audio signals.

Therefore, the `noise_generator` function is used to generate a noisy version of our dataset. This is the overlay of the original audio data and the noisy version of the audio data. `test_data_ae[0]` and `test_data_ae[10]` are used for presentation:



**Figure 1.** Comparison of the original audio and the noisy audio

### 3.3.3 Encoder and Decoder Layer

The encoder is fundamental to dimensionality reduction and feature extraction. The encoder layer in this autoencoder model is used to shrink the data set to a lower dimension, resulting in a compressed representation of the original data. This compression is useful for extracting audio features from the data set. It is typically designed to learn a mapping function from the higher dimensional input space to a lower dimensional latent space, so the most important and informative features of our audio files, in this case the bird sounds, would be captured in a compressed form.

The role of the decoder in a autoencoder model is to reconstruct the input data by mapping the low-dimensional representation obtained from the encoder back to the original input space. The structure and parameters of the decoder are optimised by training the self-encoder with reconstruction errors, allowing the autoencoder to learn effective ways of compressing and reconstructing the data during the training process.

In this autoencoder model, an encoder layer and a decoder layer are used to construct the overall structure.

### 3.3.4 Training Process

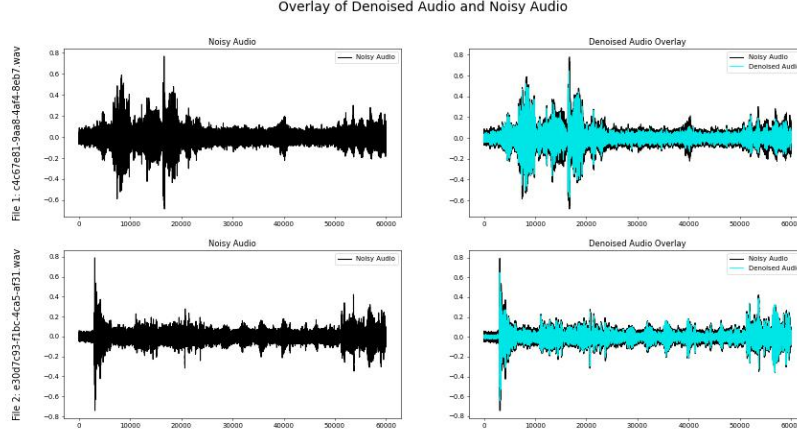
When running the model with cuda acceleration and a batch size of 8 and set audio length restriction to 60000, each epoch takes around 18-20ms per step. This table shows the recording of one of a training process, the number of epoch is 30, the batch size is set to 8:

| Epoch | Training Loss | Validation Loss | Time per step |
|-------|---------------|-----------------|---------------|
| 1     | 0.0050        | 0.0034          | 1s 68ms       |
| 2     | 0.0026        | 0.0017          | 0s 34ms       |
| 3     | 0.0021        | 0.0011          | 0s 33ms       |
| 4     | 0.0017        | 8.1263e-04      | 0s 32ms       |
| 5     | 0.0015        | 6.9204e-04      | 0s 32ms       |
| 6     | 0.0016        | 6.4425e-04      | 0s 33ms       |
| ....  |               |                 |               |
| 25    | 7.9702e-04    | 6.8721e-04      | 0s 33ms       |
| 26    | 7.4651e-04    | 3.4561e-04      | 0s 34ms       |
| 27    | 7.1640e-04    | 3.4561e-04      | 0s 33ms       |
| 28    | 6.9142e-04    | 3.1128e-04      | 0s 35ms       |
| 29    | 6.8721e-04    | 3.3617e-04      | 0s 37ms       |
| 30    | 6.8721e-04    | 3.4058e-04      | 0s 35ms       |

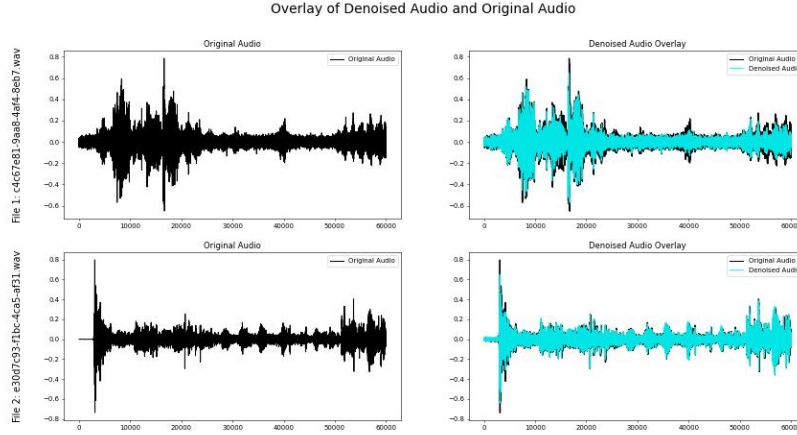
**Table 2.** A Recording of Training Process of Autoencoder Model

### 3.3.5 Prediction & Comparison

I have used different colours to show the noise reduction and the other two sounds separately. In the diagram on the right, the effect of the noise reduction can be seen visually in the black edges around the blue waveform. `test_data_ae[0]` and `test_data_ae[10]` are used for presentation:



**Figure 2.** Comparison of the denoised audio and the noisy audio



**Figure 3.** Comparison of the original audio and the denoised audio

### 3.3.6 Evaluation

The BSS evaluation criteria [3] are used for evaluation for the separation quality. There are some packages that use these criteria to allow users to perform evaluation on their data. `mir_eval.separation.bss_eval_sources` is a implementation of the BSS\_EVAL Matlab toolbox, but this seemed too slow to compute the result.

Another package is called `fast-bss-eval` [2], which is best known for its fast computation speed<sup>4</sup> compared to others. The evaluation step of this task will therefore use this package.

4. <https://pypi.org/project/fast-bss-eval/>

In order to assess the performance of my model, I compared the results obtained at different epochs during training. Specifically, I calculated the Source-to-Distortion Ratio (SDR), Signal-to-Interference Ratio (SIR), Signal-to-Artifacts Ratio (SAR), and Permutation (Perm) metrics for this autoencoder model at epoch 30 and epoch 40.

At epoch 30, this model achieved SDR, SIR, SAR, and Perm values of 10.425, 18.084, 11.719, 9.5 respectively. However, with further training until epoch 40, I observed significant improvements in the performance of our model, with the SDR, SIR, SAR, and Perm values increasing to 11.899, 19.462, 13.129, 9.5 respectively. These results suggest that increasing the number of training epochs has a positive impact on the separation performance of this model.

| Epoch Number | Mean SDR | Mean SIR | Mean SAR | Mean Perm |
|--------------|----------|----------|----------|-----------|
| 30           | 10.425   | 18.084   | 11.719   | 9.5       |
| 40           | 11.899   | 19.462   | 13.129   | 9.5       |

**Table 3.** The BBS evaluation result

### 3.4 The CNN Classifier Model

#### 3.4.1 Overview

```
CnnModel(
  (conv1): Conv2d(1, 64, kernel_size=(5, 5), stride=(1, 1), bias=False)
  (conv2): Conv2d(64, 128, kernel_size=(5, 5), stride=(1, 1), bias=False)
  (conv3): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), bias=False)
  (fc1): Linear(in_features=256, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=1, bias=True)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  (bn3): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)
```

**Table.** The Architecture of the CNN Model

## 4 How Two Models Connect

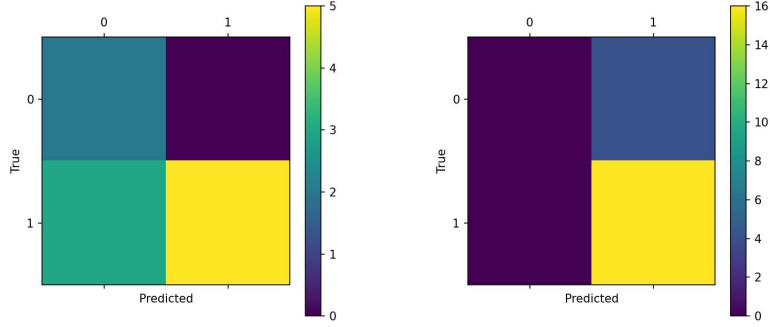
As we intuitively thought, a recording with background noise removed tends to perform better when fed into a classification network. If we use the same pre-trained model to perform the classification task, we will get a higher accuracy rate.

The two models are linked in this way, with the denoised version of the autoencoder model's output then fed as input to the same pre-trained CNN model, along with the unaltered original test data.

### 4.1 Evaluation

The confusion matrix of the denoised audio input compared to the confusion matrix of the original audio input indicates that the former has a better predictive performance in the binary classification CNN model.

For metrics such as precision, recall and F1-score, the denoised audio input only resulted in a higher recall value than the original audio input. A higher recall score indicates that the denoised audio input was better at capturing the positive instances. This suggests that the denoised audio input may have reduced the effect of noise or other audio artefacts, resulting in a better ability to detect bird audio samples, without necessarily affecting other metrics such as precision and F1-score.



**Figure 4.** Confusion matrix for original audio input and denoised audio input.

| Input                | Precision | Recall | F1-score | Accuracy |
|----------------------|-----------|--------|----------|----------|
| Original Audio Input | 1.00      | 0.62   | 1.00     | 0.70     |
| Denoised Audio Input | 0.80      | 1.00   | 0.89     | 0.80     |

**Table 4.** Comparison

## 5 Discussion

The result of the evaluation suggests that denoising the audio input may be beneficial for certain aspects of the classification task, such as reducing the impact of noise or artefacts, but may not necessarily lead to improvements in all performance metrics. Further analysis and experimentation may be required to better understand the impact of denoising on the specific bird audio classification task and the trade-offs involved.

Some possible improvements include increasing the number of encoder and decoder layers instead of using only one encoder and decoder layer. The approach which proposed by Wu et al. [5], called *Only-Noisy Training (ONT)* is designed for an effective complex-valued speech denoising network. They use five coding layers and five decoding layers together in their DCUnet10\_TSTM model.

## Bibliography

- [1] Tom Denton, Scott Wisdom, and John R. Hershey. Improving bird classification with unsupervised sound separation. 2021.
- [2] Robin Scheibler. Sdr — medium rare with fast computations. 2021.
- [3] Fabian-Robert Stöter, Antoine Liutkus, and Nobutaka Ito. The 2018 signal separation evaluation campaign. 2018.
- [4] Dan Stowell, Yannis Stylianou, Mike Wood, Hanna Pamuła, and Hervé Glotin. Automatic acoustic detection of birds through deep learning: the first bird audio detection challenge. 2018.
- [5] Jiasong Wu, Qingchun Li, Guanyu Yang, Lei Li, Lotfi Senhadji, and Huazhong Shu. Self-supervised speech denoising using only noisy audio signals. 2023.